Let's create a copy of our development config as a starting point for our production config:

```
$ cp -r .env/development .env/production
```

This should leave us with the following file structure:

```
$ tree .env
.env
├── development
│   ├── database
│   └── web
└── production
    ├── database
    └── web

2 directories, 4 files
```

Now let's edit the .env/production/web file to look like this:

```
DATABASE_HOST=database
RAILS_ENV=production
SECRET_KEY_BASE=
RAILS_LOG_TO_STDOUT=true
RAILS_SERVE_STATIC_FILES=true
```

We have to remember to set RAILS_ENV to production so that the app starts in production mode.

Rails uses SECRET_KEY_BASE as a security mechanism to sign the cookies it sets, allowing it to verify that cookies it receives can be trusted. We've intentionally left it blank because we need to generate a new one for our production config. Let's do that now using Rails' handy Rake task:

```
$ docker-compose exec web bin/rails secret
9d6d05e1990f81bbba154b1fc54c23c6ffaafb081a07e5ac3731160a6126e711c2f1d7623b6d
5140c03686462cda966344fe6f1b35fa44c14eaba50769692c74
```

Paste your generated secret key into your .env/production/web file as the value for the SECRET_KEY_BASE environment variable.

By default, Rails logs to a file on disk in logs/<environment>.log. In a Docker environment, we don't want the logs to be written to the filesystem *inside* the container as they're hard to get to from there. Instead, we want to configure the app to output its logs to stdout, which allows us to view the log using Docker's log commands. We can do this by setting the RAILS_LOG_TO_STDOUT environment variable (added in Rails 5) to true.

Also, we are going to have Rails serve our static assets, so we need to set RAILS_SERVE_STATIC_FILES to true to make them available in production.